WiFi 信号分析仪 — 项目原理与调用关系分析

> 目标读者：初次接触本项目的开发者
> 平台：ESP32-S3-WROOM-1 + SSD1306 OLED
> 框架：ESP-IDF v5.1+（FreeRTOS）

---P

目录

1. 项目概述
2. 硬件架构
3. 软件模块划分
4. 启动流程
5. 任务模型（FreeRTOS）
6. 模块详解
    - wifi_scanner — WiFi 扫描模块
    - oled_display — OLED 显示模块
    - web_server — Web 服务器模块
7. 数据流向
8. 函数调用关系图
9. 关键数据结构
10. 线程安全机制
11. Web 前端原理
12. 文件结构速查

---

1. 项目概述

本项目将 ESP32-S3 变成一台便携式 WiFi 信号分析仪，具备三种交互方式：

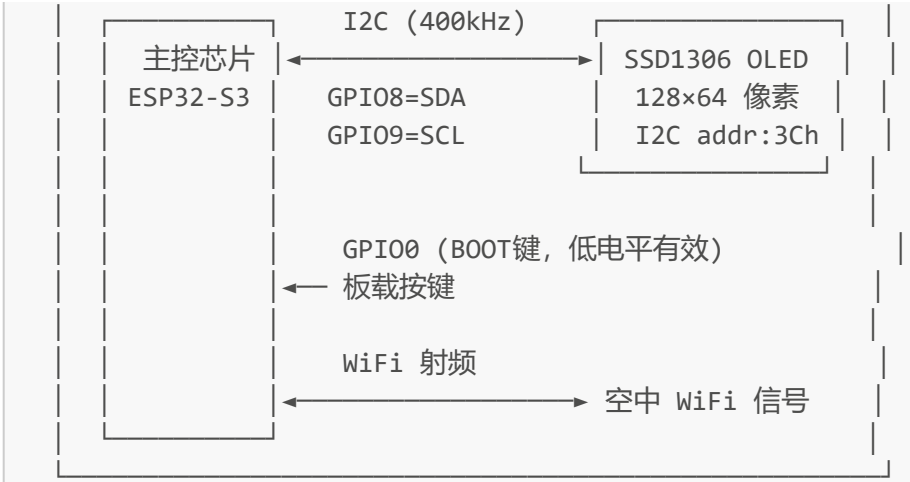
交互方式	说明
OLED 屏幕	实时显示周边 WiFi 列表（信号条 + RSSI 值），每页 4 条，按 BOOT 键翻页
Web 界面	手机/电脑连接设备热点后，浏览器访问可视化页面，含信道占用图
串口日志	每次扫描结果通过 UART 输出，便于调试

核心工作流程（一句话）：每 10 秒扫描一次周边 WiFi → 结果同时推送给 OLED 显示模块和 Web 服务器模块。

---

2. 硬件架构



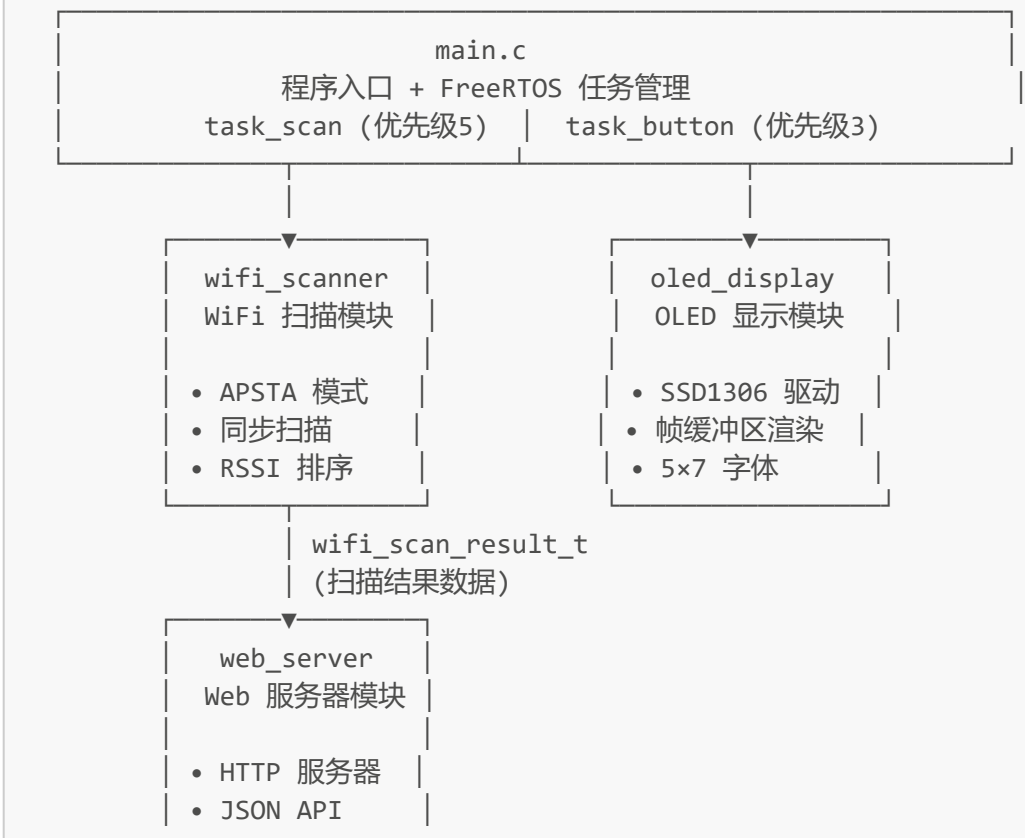


接线说明:

ESP32-S3		SSD1306 OLED
3.3V	——→	VCC
GND	——→	GND
GPIO8	——→	SDA
GPIO9	——→	SCL

---

3. 软件模块划分



```
        │ • 嵌入式 HTML   │
        └─────────────────┘
```

---

4. 启动流程

```
app_main()
    │
    ├─ nvs_flash_init()          ← 初始化 NVS（非易失存储，WiFi 配置依赖）
    │
    ├─ esp_netif_init()          ← 初始化网络接口层
    ├─ esp_event_loop_create_default()  ← 创建系统事件循环
    │
    ├─ oled_display_init()       ← 初始化 I2C + SSD1306（失败不影响 Web 功能）
    │       │
    │       ├─ i2c_param_config()
    │       ├─ i2c_driver_install()
    │       └─ ssd1306_init_sequence()  ← 发送 17 条初始化命令
    │
    ├─ wifi_scanner_init()       ← 初始化 WiFi（APSTA 模式）
    │       │
    │       ├─ esp_netif_create_default_wifi_ap()
    │       ├─ esp_netif_create_default_wifi_sta()
    │       ├─ esp_wifi_init()
    │       ├─ esp_wifi_set_mode(WIFI_MODE_APSTA)
    │       ├─ esp_wifi_set_config()   ← 配置 AP: SSID=WiFi-Analyzer
    │       └─ esp_wifi_start()
    │
    ├─ web_server_init()         ← 启动 HTTP 服务器
    │       │
    │       ├─ xSemaphoreCreateMutex()  ← 创建数据保护互斥锁
    │       ├─ httpd_start()
    │       ├─ 注册路由 GET /
    │       └─ 注册路由 GET /api/scan
    │
    ├─ oled_display_show_boot()  ← 显示启动画面（AP 信息）
    ├─ vTaskDelay(3000ms)        ← 等待 3 秒
    │
    ├─ xTaskCreate(task_scan,   优先级5，栈4096)  ← 创建扫描任务
    └─ xTaskCreate(task_button, 优先级3，栈4096)  ← 创建按键任务
```

---

5. 任务模型（FreeRTOS）

系统运行时共有 **2 个用户任务** 并发执行：

```
时间轴 ──────────────────────────────────────────────────────►

task_scan（优先级5）
```

```
|←──────── 扫描 ~1-3s ────────→|←── 等待 10s ──→|←── 扫描 ──→|
| wifi_scanner_scan()          | vTaskDelay()   |            |
| oled_display_update()        |                |            |
| web_server_update_data()     |                |            |

task_button (优先级3)
|poll|poll|poll|poll|poll|poll|poll|poll|poll|poll|poll|...
  20ms间隔轮询 GPIO0 电平，检测下降沿 + 50ms 消抖
```

任务名	优先级	栈大小	职责
task_scan	5（高）	4096 B	周期性 WiFi 扫描，更新 OLED 和 Web 数据
task_button	3（低）	4096 B	轮询 BOOT 键，触发 OLED 翻页

按键消抖逻辑：

```
检测到下降沿（高→低）
    └─ vTaskDelay(50ms)      ← 等待抖动结束
        └─ 再次读取 GPIO0
            └─ 仍为低电平 → 确认按下 → oled_display_next_page()
```

---

6. 模块详解

6.1 wifi_scanner — WiFi 扫描模块

工作模式：APSTA（AP + STA 同时运行）

```
                    ┌──────────────────────────────┐
                    │          ESP32-S3            │
  手机/电脑         │                              │
  连接热点 ────────→│  AP 接口 (WiFi-Analyzer)     │
  192.168.4.1       │  提供 Web 服务热点           │
                    │                              │
  周边 WiFi 网络    │  STA 接口                    │
  ←─────────────────│  主动扫描，不连接任何网络    │
                    └──────────────────────────────┘
```

扫描流程：

```
wifi_scanner_scan()
    │
    ├─ memset(result, 0)          ← 清空结果缓冲区
    │
    ├─ esp_wifi_scan_start()      ← 同步扫描（阻塞 ~1-3 秒）
    │   scan_cfg:
```

```
    │     channel = 0              ← 扫描全部信道 (1-13)
    │     show_hidden = true       ← 包含隐藏网络
    │     scan_type = ACTIVE       ← 主动扫描 (发送 Probe Request)
    │     active.min = 100ms
    │     active.max = 300ms
    │
    ├─ esp_wifi_scan_get_ap_num() ← 获取发现的 AP 数量
    │
    ├─ calloc(fetch_count)         ← 分配临时缓冲区
    ├─ esp_wifi_scan_get_ap_records()  ← 获取原始扫描结果
    │
    ├─ 转换为 wifi_network_t 结构  ← 复制 SSID/RSSI/channel/auth/BSSID
    │
    ├─ free(ap_records)            ← 释放临时缓冲区
    │
    └─ sort_by_rssi()              ← 冒泡排序，按 RSSI 降序 (强→弱)
```

RSSI 转信号格数:

RSSI 范围	信号格数	信号质量
≥ -60 dBm	4 格	优秀
-70 ~ -61 dBm	3 格	良好
-80 ~ -71 dBm	2 格	一般
< -80 dBm	1 格	较差

6.2 oled_display — OLED 显示模块

分层架构:

```
┌──────────────────────────────────────────────┐
│          应用层 (显示逻辑)                    │
│  oled_display_show_boot()                    │
│  oled_display_update()                       │
│  oled_display_next_page()                    │
├──────────────────────────────────────────────┤
│          图形层 (帧缓冲区操作)                │
│  fb_clear()      fb_set_pixel()              │
│  fb_hline()      fb_fill_rect()              │
│  fb_draw_rect()  fb_draw_char()              │
│  fb_draw_str()   fb_draw_printf()            │
│  fb_draw_signal_bars()                       │
├──────────────────────────────────────────────┤
│          帧缓冲区 (1024 字节 RAM)             │
│  s_framebuf[128 × 8 pages]                   │
│  每字节 = 1 列 × 8 行 (bit0=顶, bit7=底)      │
├──────────────────────────────────────────────┤
│          SSD1306 驱动层 (I2C)                │
│  ssd1306_cmd()   ssd1306_data()              │
```

```
│  ssd1306_flush() ssd1306_init_sequence()    │
├─────────────────────────────────────────────┤
│           ESP-IDF I2C 驱动                  │
│  i2c_master_write_to_device()               │
│  i2c_master_cmd_begin()                     │
└─────────────────────────────────────────────┘
```

帧缓冲区内存布局：

```
s_framebuf[1024 字节]

Page 0 (y=0~7):   [col0][col1][col2]...[col127]   ← 128 字节
Page 1 (y=8~15):  [col0][col1][col2]...[col127]   ← 128 字节
...
Page 7 (y=56~63): [col0][col1][col2]...[col127]   ← 128 字节

索引公式: idx = (y/8) * 128 + x
位操作:   framebuf[idx] |= (1 << (y%8))  ← 点亮像素
```

OLED 屏幕布局（128×64 像素）：

```
y=0  ┌──────────────────────────────────────────────────────────────┐
     │ WiFi Analyzer                                    N:12        │ ← 标题栏
y=9  ├──────────────────────────────────────────────────────────────┤ ← 分隔线
y=14 │ MyHomeWiFi                                ▓▓▓▓  -45          │ ← 网络1
y=24 │ ChinaNet-xxxx                             ▓▓▓░  -68          │ ← 网络2
y=34 │ TP-LINK_5G                                ▓▓░░  -75          │ ← 网络3
y=44 │ <hidden>                                  ▓░░░  -88          │ ← 网络4
y=54 ├──────────────────────────────────────────────────────────────┤ ← 分隔线
y=56 │ P1/3  Best:CH6 -45dBm                                        │ ← 状态栏
y=63 └──────────────────────────────────────────────────────────────┘
     x=0                                                    x=127
```

翻页机制：

```
s_display_page（静态变量）
s_last_result（缓存最近一次扫描结果）

oled_display_next_page()
    └─ s_display_page = (s_display_page + 1) % total_pages
        └─ oled_display_update(NULL)   ← 传 NULL = 使用缓存数据重绘
```

刷新流程（每次调用 `ssd1306_flush()`）：

```
1. 发送命令：设置列地址范围 0~127
2. 发送命令：设置页地址范围 0~7
3. 通过 I2C 发送全部 1024 字节帧缓冲区
    (I2C 地址 0x3C, 控制字节 0x40 表示数据模式)
```

6.3 web_server — Web 服务器模块

HTTP 路由：

路由	方法	处理函数	返回内容
`/`	GET	`handler_root()`	`index.html`（编译时嵌入固件）
`/api/scan`	GET	`handler_scan_api()`	JSON 格式扫描结果

HTML 嵌入原理：

```
# main/CMakeLists.txt 中配置
idf_component_register(
    EMBED_TXTFILES "index.html"   ← ESP-IDF 编译时将文件内容嵌入固件
)
```

编译后链接器自动生成两个符号：

```
extern const char index_html_start[] asm("_binary_index_html_start");
extern const char index_html_end[]   asm("_binary_index_html_end");
// 文件大小 = index_html_end - index_html_start
```

JSON 响应格式：

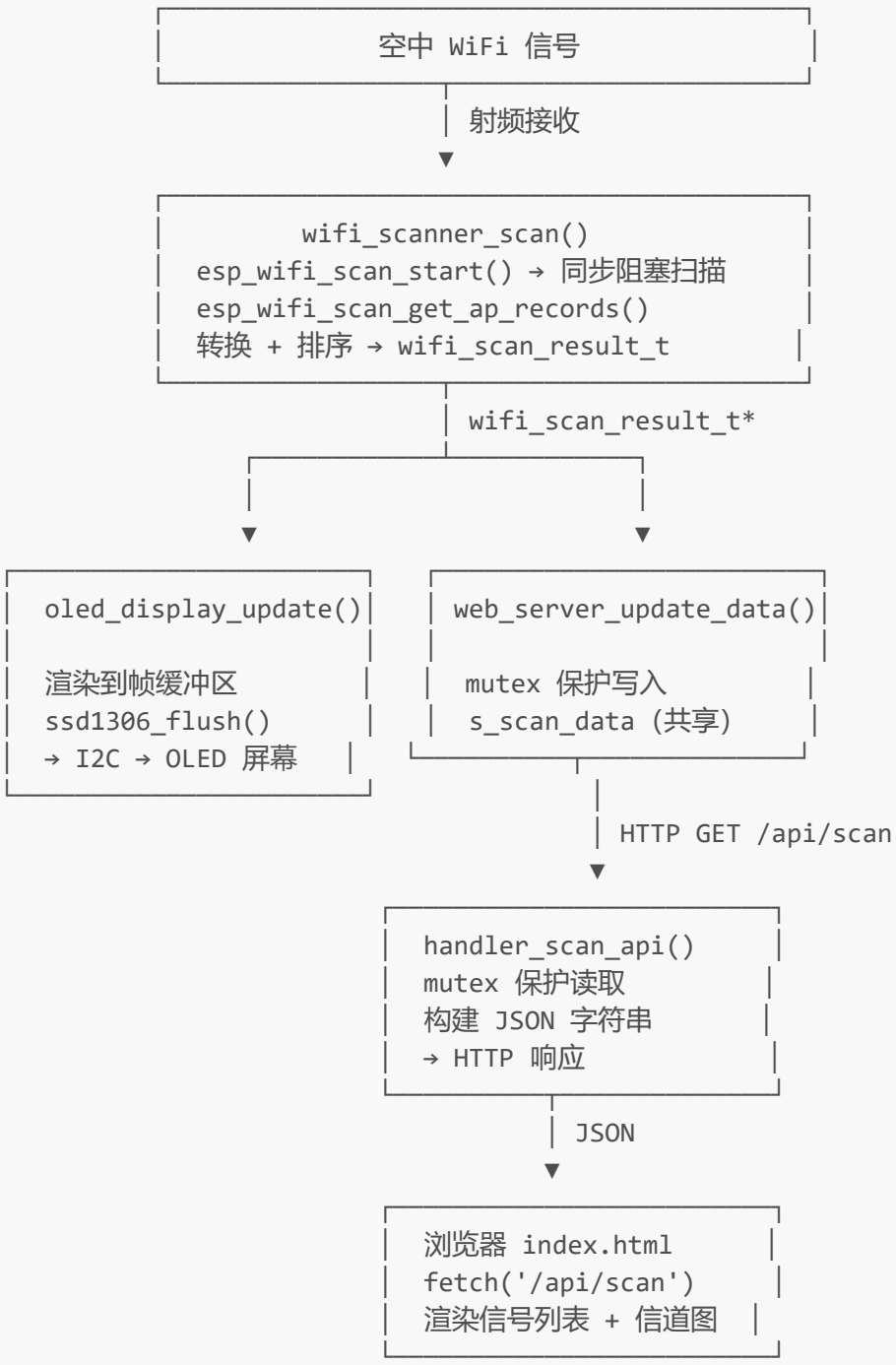
```
{
  "networks": [
    {
      "ssid": "MyHomeWiFi",
      "rssi": -45,
      "channel": 6,
      "auth": "WPA2",
      "open": false,
      "bssid": "AA:BB:CC:DD:EE:FF"
    },
    ...
  ]
}
```

线程安全数据更新:

```
task_scan 线程                    HTTP 服务器线程 (httpd 内部任务)
    │                                    │
    │ web_server_update_data(&result)    │ handler_scan_api()
    │   xSemaphoreTake(mutex, 500ms)     │   xSemaphoreTake(mutex, 500ms)
    │   memcpy(&s_scan_data, result)     │   读取 s_scan_data 构建 JSON
    │   xSemaphoreGive(mutex)            │   xSemaphoreGive(mutex)
    │                                    │
```

---

7. 数据流向



---

8. 函数调用关系图

8.1 初始化阶段

```
app_main()
├── nvs_flash_init()                          [ESP-IDF]
├── esp_netif_init()                          [ESP-IDF]
├── esp_event_loop_create_default()           [ESP-IDF]
│
├── oled_display_init()                       [oled_display.c]
│   ├── i2c_param_config()                    [ESP-IDF driver]
│   ├── i2c_driver_install()                  [ESP-IDF driver]
│   └── ssd1306_init_sequence()               [oled_display.c 内部]
│       └── ssd1306_cmd() × 17               [oled_display.c 内部]
│           └── i2c_master_write_to_device()  [ESP-IDF driver]
│
├── wifi_scanner_init()                       [wifi_scanner.c]
│   ├── esp_netif_create_default_wifi_ap()    [ESP-IDF]
│   ├── esp_netif_create_default_wifi_sta()   [ESP-IDF]
│   ├── esp_wifi_init()                       [ESP-IDF]
│   ├── esp_wifi_set_mode(APSTA)              [ESP-IDF]
│   ├── esp_wifi_set_config()                 [ESP-IDF]
│   └── esp_wifi_start()                      [ESP-IDF]
│
├── web_server_init()                         [web_server.c]
│   ├── xSemaphoreCreateMutex()               [FreeRTOS]
│   ├── httpd_start()                         [ESP-IDF]
│   ├── httpd_register_uri_handler(GET /)     [ESP-IDF]
│   └── httpd_register_uri_handler(GET /api/scan) [ESP-IDF]
│
├── oled_display_show_boot()                  [oled_display.c]
│   ├── fb_clear()
│   ├── fb_draw_str() × 6
│   ├── fb_hline()
│   └── ssd1306_flush()
│       ├── ssd1306_cmd() × 6
│       └── ssd1306_data()
│           └── i2c_master_cmd_begin()        [ESP-IDF driver]
│
├── xTaskCreate(task_scan, 优先级5)           [FreeRTOS]
└── xTaskCreate(task_button, 优先级3)         [FreeRTOS]
```

8.2 运行阶段 — task_scan（每 10 秒循环）

```
task_scan()  [无限循环]
└── wifi_scanner_scan(&result)                [wifi_scanner.c]
    ├── esp_wifi_scan_start()                 [ESP-IDF, 阻塞]
```

```
    ├── esp_wifi_scan_get_ap_num()            [ESP-IDF]
    ├── calloc()
    ├── esp_wifi_scan_get_ap_records()        [ESP-IDF]
    ├── [数据转换循环]
    │   └── wifi_auth_mode_str()              [wifi_scanner.c]
    ├── free()
    └── sort_by_rssi()                        [wifi_scanner.c 内部]
        │
        ▼ (扫描成功后)
        ├── oled_display_update(&result)      [oled_display.c]
        │   ├── fb_clear()
        │   ├── fb_draw_str()                 ← 标题 "WiFi Analyzer"
        │   ├── fb_draw_printf()              ← 网络数量 "N:xx"
        │   ├── fb_hline()                    ← 分隔线
        │   ├── [网络列表循环，每页4条]
        │   │   ├── fb_draw_str()             ← SSID
        │   │   ├── fb_draw_signal_bars()     ← 信号条
        │   │   │   ├── fb_fill_rect()        ← 实心格（有信号）
        │   │   │   └── fb_draw_rect()        ← 空心格（无信号）
        │   │   └── fb_draw_printf()          ← RSSI 数值
        │   ├── fb_hline()                    ← 底部分隔线
        │   ├── fb_draw_printf()              ← 状态栏
        │   └── ssd1306_flush()               ← 刷新到屏幕
        │
        └── web_server_update_data(&result)   [web_server.c]
            ├── xSemaphoreTake(mutex)         [FreeRTOS]
            ├── memcpy(&s_scan_data, result)
            └── xSemaphoreGive(mutex)         [FreeRTOS]
```

8.3 运行阶段 — task_button（每 20ms 轮询）

```
task_button()  [无限循环，20ms 间隔]
└── gpio_get_level(GPIO0)
    └── [检测到下降沿]
        └── vTaskDelay(50ms)                  ← 消抖
            └── gpio_get_level(GPIO0)         ← 二次确认
                └── [仍为低电平]
                    └── oled_display_next_page()  [oled_display.c]
                        ├── s_display_page = (page+1) % total_pages
                        └── oled_display_update(NULL)  ← 用缓存数据重绘
                            └── ssd1306_flush()
```

8.4 运行阶段 — HTTP 请求处理

```
[浏览器 GET /]
└── handler_root()                            [web_server.c]
    └── httpd_resp_send(index_html_start)     ← 直接发送嵌入的 HTML

[浏览器 GET /api/scan]
```

```
└── handler_scan_api()                          [web_server.c]
    ├── xSemaphoreTake(mutex)                   [FreeRTOS]
    ├── [遍历 s_scan_data.networks]
    │   ├── [SSID 转义: " → \", \ → \\]
    │   └── snprintf() 构建 JSON 片段
    ├── xSemaphoreGive(mutex)
    └── httpd_resp_send(json_buf)
```

---

9. 关键数据结构

wifi_network_t — 单个 WiFi 网络

```c
typedef struct {
    char     ssid[33];          // SSID 字符串（最长 32 字节 + '\0'）
    int8_t   rssi;              // 信号强度（dBm），范围约 -100 ~ 0
    uint8_t  channel;           // 信道号（1~13 for 2.4GHz）
    wifi_auth_mode_t auth_mode; // 加密方式（OPEN/WEP/WPA/WPA2/WPA3）
    uint8_t  bssid[6];          // MAC 地址（6 字节）
} wifi_network_t;
```

wifi_scan_result_t — 一次扫描的全部结果

```c
typedef struct {
    wifi_network_t networks[10]; // 最多 10 个网络（WIFI_SCAN_MAX_RESULTS）
    uint16_t       count;        // 实际发现的网络数量
} wifi_scan_result_t;
```

模块内部状态（静态变量）

变量	所在文件	说明
`s_framebuf[1024]`	oled_display.c	OLED 帧缓冲区
`s_display_page`	oled_display.c	当前显示页码
`s_last_result`	oled_display.c	缓存的最近扫描结果（翻页用）
`s_has_result`	oled_display.c	是否已有扫描结果
`s_scan_data`	web_server.c	Web 服务器持有的扫描数据
`s_data_mutex`	web_server.c	保护 s_scan_data 的互斥锁
`s_ap_netif`	wifi_scanner.c	AP 网络接口句柄
`s_ap_ip_str`	wifi_scanner.c	AP IP 地址字符串

---

10. 线程安全机制

本项目存在一处跨任务的共享数据访问：

```
写入方：task_scan（FreeRTOS 任务）
读取方：httpd 内部任务（处理 HTTP 请求）
共享数据：s_scan_data（wifi_scan_result_t，约 330 字节）
保护方式：FreeRTOS 互斥锁（s_data_mutex）
超时时间：500ms（防止死锁）
```

注意：`oled_display_update()` 和 `oled_display_next_page()` 均在各自的任务中调用，不存在并发访问，因此 OLED 模块无需加锁。

---

11. Web 前端原理

`index.html` 是一个纯前端单页应用（无框架依赖），编译时通过 ESP-IDF 的 `EMBED_TXTFILES` 机制嵌入固件。

前端工作流程：

```
页面加载
    │
    ├── loadData()                    ← 立即执行一次
    │   ├── fetch('/api/scan')        ← 请求 JSON 数据
    │   ├── renderList(networks)      ← 渲染信号列表
    │   │   └── 按 RSSI 排序 → 生成 HTML 卡片
    │   │       每张卡片包含：锁图标、SSID、信号条、RSSI值、信道号
    │   └── renderChannelChart(networks)  ← 渲染信道占用图
    │       └── 统计 CH1~CH13 各信道网络数量和最强信号
    │           → 生成柱状图（推荐信道 1/6/11 高亮显示）
    │
    └── setInterval(loadData, 10000)  ← 每 10 秒自动刷新
```

信号强度颜色编码：

RSSI	颜色	含义
≥ -60 dBm	绿色 `#3fb950`	强信号
-75 ~ -61 dBm	黄色 `#d29922`	中等信号
< -75 dBm	红色 `#f85149`	弱信号

---

12. 文件结构速查

```
wifi_analyzer/
├── CMakeLists.txt          顶层 CMake（项目名 wifi_analyzer）
├── sdkconfig.defaults      ESP-IDF 默认配置
├── README.md               用户文档
└── main/
    ├── CMakeLists.txt      main 组件配置（含 EMBED_TXTFILES index.html）
    │
    ├── main.c              ★ 程序入口
    │                         - app_main()：初始化 + 创建任务
    │                         - task_scan()：扫描循环任务
    │                         - task_button()：按键轮询任务
    │
    ├── wifi_scanner.h      WiFi 扫描模块接口
    │                         - 数据结构定义（wifi_network_t, wifi_scan_result_t）
    │                         - 常量定义（AP SSID/密码/信道）
    ├── wifi_scanner.c      WiFi 扫描模块实现
    │                         - wifi_scanner_init()：APSTA 模式初始化
    │                         - wifi_scanner_scan()：同步扫描
    │                         - wifi_rssi_to_bars()：RSSI 转格数
    │                         - wifi_auth_mode_str()：加密方式转字符串
    │
    ├── oled_display.h      OLED 显示模块接口
    │                         - 硬件配置（I2C 引脚、地址、频率）
    │                         - 屏幕尺寸常量
    ├── oled_display.c      OLED 显示模块实现（~500行）
    │                         - SSD1306 I2C 驱动（底层）
    │                         - 帧缓冲区图形操作（中层）
    │                         - WiFi 信息显示逻辑（上层）
    │
    ├── web_server.h        Web 服务器模块接口
    ├── web_server.c        Web 服务器模块实现
    │                         - web_server_init()：启动 HTTP 服务器
    │                         - web_server_update_data()：线程安全数据更新
    │                         - handler_root()：返回 HTML 页面
    │                         - handler_scan_api()：返回 JSON 数据
    │
    └── index.html          Web 前端页面（编译时嵌入固件）
                              - 信号列表（按 RSSI 排序）
                              - 2.4G 信道占用柱状图
                              - 每 10 秒自动刷新
```

附：配置常量速查

常量	值	说明
WIFI_AP_SSID	"WiFi-Analyzer"	AP 热点名称
WIFI_AP_PASSWORD	"12345678"	AP 热点密码
WIFI_AP_CHANNEL	6	AP 工作信道

常量	值	说明
WIFI_AP_MAX_CONN	4	AP 最大连接数
WIFI_SCAN_MAX_RESULTS	10	最多保存扫描结果数
SCAN_INTERVAL_MS	10000	扫描间隔（10 秒）
PAGE_BTN_GPIO	GPIO_NUM_0	翻页按键（板载 BOOT）
OLED_SDA_GPIO	8	I2C SDA 引脚
OLED_SCL_GPIO	9	I2C SCL 引脚
OLED_I2C_ADDR	0x3C	SSD1306 I2C 地址
OLED_I2C_FREQ_HZ	400000	I2C 时钟频率（400kHz）
OLED_WIDTH	128	OLED 宽度（像素）
OLED_HEIGHT	64	OLED 高度（像素）